

Homework #1: Zag Farkle Rolling and Scoring

CPSC 224 - Software Development - Gonzaga University – Fall 2026

Contents

RUNDOWN OF THE PROJECT	1
THE PROBLEM	1
PROJECT REQUIREMENTS	2
NOTE ABOUT EDITING CODE	2
USER INTERFACE NOTE	2
OTHER GENERAL REQUIREMENTS	2
USING GIT AND GITHUB.....	3
SUBMISSION DETAILS	5
IMPORTANT NOTE ON INTEGRITY	6
GRADING RUBRIC.....	6
APPENDIX A: TERMINOLOGY AND CLARIFICATIONS	7
APPENDIX B: MELD AND COMBO SCORING SPECIFICATION FOR ZAG FARKLE	8

RUNDOWN OF THE PROJECT

THE PROBLEM

Produce a Java implemented equivalent of the crandallFarkle.cpp C++ program available here:

<https://github.com/DrewBogdan/farkle>

You will be using the template repository provided on the GitHub. Click this link and then click the green “Use this template” button in the top right of GitHub: <https://github.com/Gonzaga-Fall-2026-CPSC-224/HW1>

IMPORTANT: To receive a grade on your homework you must add Abby (abbyfewel) and Jon (jlarson023) as collaborators to your GitHub Repo.

You will be using this repository for almost every homework assignment this semester and tagging each completed assignment, more on tagging in a later section.

Execution and output of your solutions should be roughly identical to the C++ version, an example of which is shown below. That doesn’t mean that you should just copy the C++ code verbatim, even though that would work relatively well in Java. You should look at the desired game behaviors and consider an object-oriented approach to building up the same results as the example C++ code. Modeling parts of the game, such as Die, Hand of dice, Melds, Scorecards, etc should be strongly considered. Also be aware that your main() method should probably be no longer than 10 lines of code when you are done, which is in sharp contrast to the example code given.

The classes you produce to complete this assignment will potentially be used in subsequent individual assignments and will be part of the toolkit you bring to your team for our group project.

We are using the basic Farkle rules as shown here, with our custom scoring as defined in Appendix B of this document: <https://gathertogethergames.com/farkle>

Scoring for our Zag Farkle game is a simple form of combos. These are a subset of the full set that are shown in many versions of the game that you will find online. These combinations (combos) are shown in *Appendix A* at the end of this assignment description.

An online example of the game (with slightly different scoring) can be found and played here: <https://cardgames.io/farkle/>

PROJECT REQUIREMENTS

For this assignment, you need to implement these features, as you see in the C++ reference implementation of the game. You should have your game act as much like the C++ game as you can, especially the menu interface.

- Single player - only a single player playing the game
- Single turn - you do not have to do multiple rounds
- Roll the six dice
- Show the hand of dice rolled to the player
- Detect if the player rolled a Farkle and end their turn if they rolled a Farkle
- If not a Farkle, ask the player which dice they would like to add to their meld (a set of scoring dice)
- When the player is done adding dice to the meld, show the total score the player got
- Show both the dice used in the meld, and the dice unused

NOTE ABOUT EDITING CODE

Start by making a file called `Farkle.java`, and creating your `main()` method in there. More classes (java files) will need to be added eventually as the project grows; what those are and how many is completely up to your design.

You will be creating tests in the `\test` folder later on in the semester as well. Feel free to create additional class files in the folder.

This class is all about object oriented programming, it is required that you split up your program into not only the files provided, but additional classes/files as you see fit

Have fun! If you want to change the UI or interactivity, feel free! Just make sure you provide well documented instructions during the game so we will know how to run and play it.

USER INTERFACE NOTE

You should follow the example C++ code's user interface. This means it should look and act the same way for showing the dice, the current meld, the menu options, score, and how dice are added/removed from the meld. Pay special attention to an A through F menu style for the meld.

This assignment is intended to have you work through most of the java basics found in our textbook. One hint: The way you store a hand might be more flexible down the road if you investigate and use a generic `ArrayList`¹ instead of hard coding things with arrays. This is not required, but may save you some rework later.

OTHER GENERAL REQUIREMENTS

- 1) You need to edit a `README.md` file in the root of your repository. Add your name and make sure the semester is correct. Then, have it say "Current assignment: HW1". Future assignments should update that as you go through all of them.
- 2) When you commit and push your code to GitHub, it will build, test, check, and bundle up your work using GitHub Actions. This is already setup for the repository, so you don't have to make it run. What you will be graded on (see Grading section) is on whether your code conforms to the style checker, tests, and spell checker. As long as they all keep passing properly (easier to do than you might think - we'll talk in class about it), then you get full marks, but to get a perfect score, they have to all keep working.

¹ https://www.w3schools.com/java/java_arraylist.asp

USING GIT AND GITHUB

Step 1: Make sure your JDK environment is set up correctly and you can run java files.

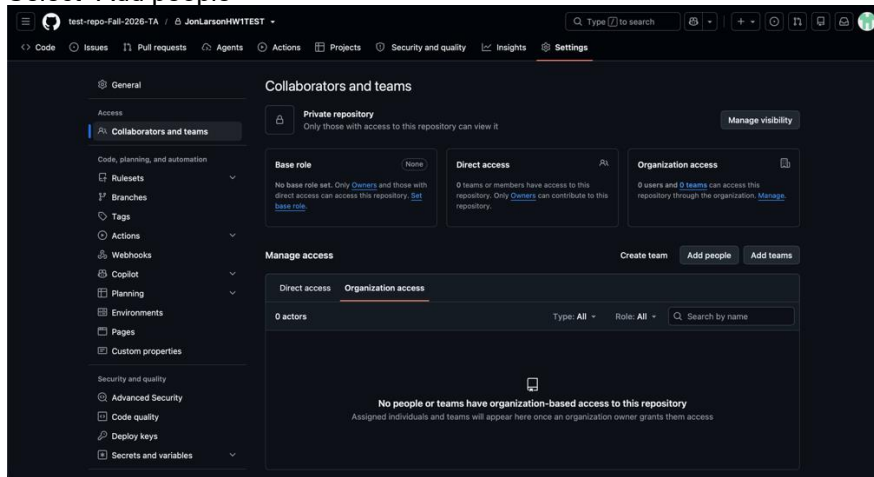
- If you are using VSCode, make sure you have the Java Extensions downloaded. **This should've been done in the first week of class, and you should be good to go!**

Step 2: Use the GitHub Repo Template – <https://github.com/Gonzaga-Fall-2026-CPSC-224/HW1>

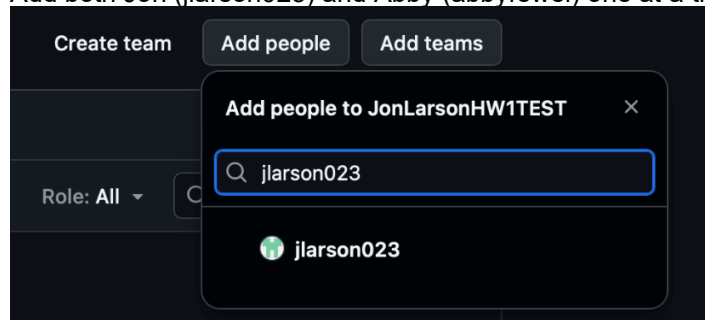
- Select “Use this template”
- Name your repo using the following naming convention: “FirstLastHW1”
- Create Repository

Step 3: Add collaborators

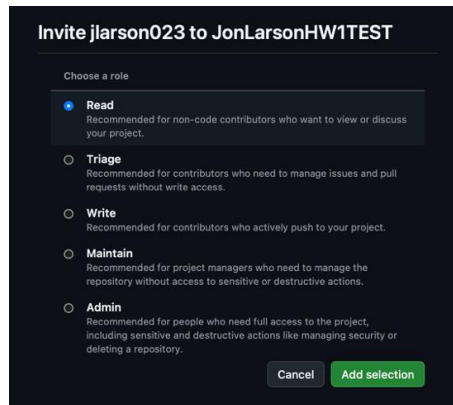
- Select “Settings” for your repo
- For each collaborator:
 - o Select “Add people”



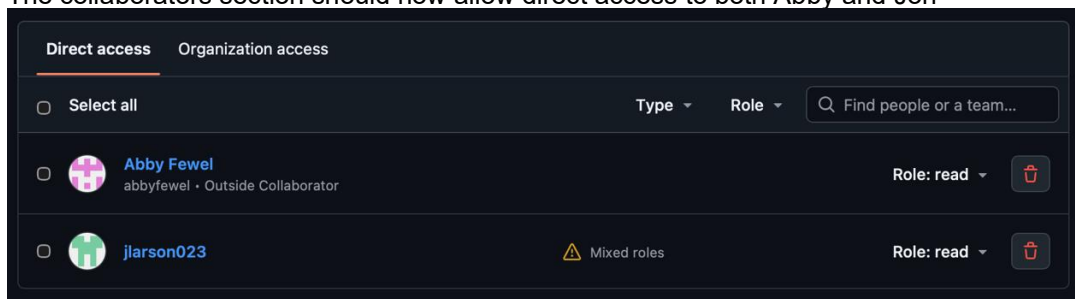
- Add both Jon (jlarson023) and Abby (abbyfewel) one at a time



- Choose a role:
 - Read is all that is necessary



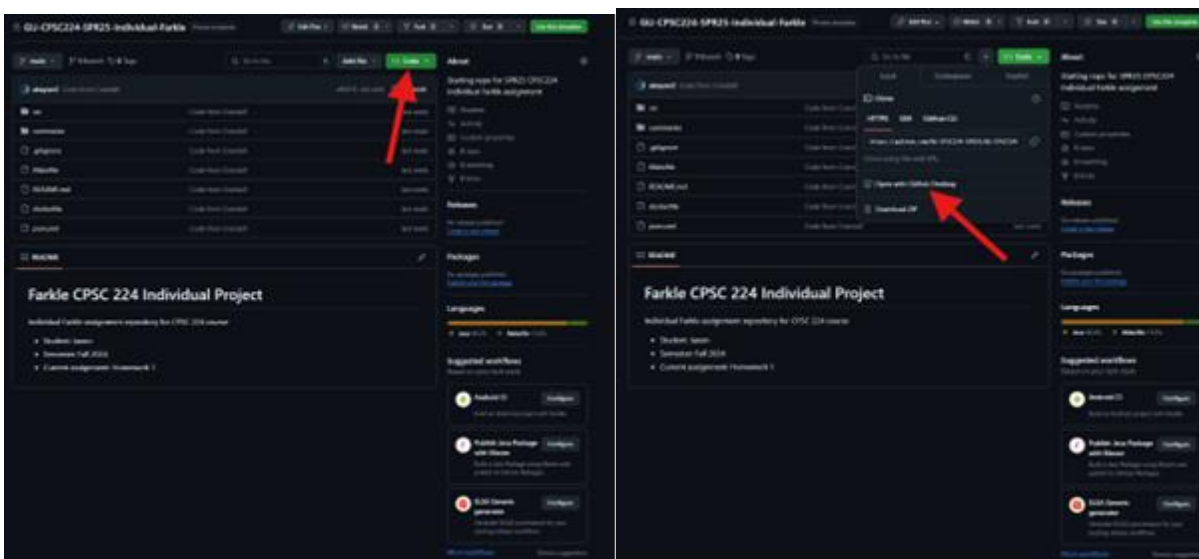
- Click “Add selection”
- The collaborators section should now allow direct access to both Abby and Jon



Step 4: Clone the repository to your local machine

- Windows and Mac - Two Options:
 - Use command line: Set up Git - GitHub Docs
 - **RECOMMENDED:** Use GitHub Desktop: GitHub Desktop | Simple collaboration from your desktop'
- Linux must use command line: Set up Git - GitHub Docs
- **If you chose to use GitHub desktop**, follow these screenshots to clone a repository once you have the application (like VS Code) downloaded and your GitHub account logged into it.

1: Go to the repository you just finished adding collaborators to and click the button that says “Code”



2: Make sure HTTPS is selected, and copy the URL.

3: Open your Integrated Development Environment (IDE), ideally VS Code, and open a new window (in VS Code, you can click File → New Window).

4: In VS Code, there is a button to click that says “Clone Git Repository”, select that.

5: Paste the URL that you copied from GitHub and click Enter.

You should now be good to go! From here, you can code and make commits/pushes/tags. More on that is below.

Making Git Commits:

- There are ways to commit in the IDE using a GUI, and there are ways to commit using the terminal.
- To make commits from the terminal, make sure you are in the correct folder.
 - o Type “git add .” which adds every file to the staging area to commit
 - o Type “git commit -m “here is a commit message”” and briefly describe your commit in that message.
 - o Type “git push” to push your commits to GitHub.
 - o When you are tagging the version of your code that you want to be graded for each homework, type “git tag -a “HW1” -m “Finished Homework 1””, or whatever homework submission you are tagging. Then, type “git push --tags” to push that tag to GitHub.
 - o **Here's more info on Git Tag is you want to know more details: <https://git-scm.com/book/en/v2/Git-Basics-Tagging>**

The TA/grader can then clone your repo and checkout exactly the version you want graded for HW1. As we continue through HW2 and beyond, each assignment will be tagged so you can see the progress from one assignment to the next.

SUBMISSION DETAILS

- ALL Farkle homework assignments will be submitted as a repo link on Canvas; you will not need to submit any code files specifically to Canvas.
- Each homework submission commit MUST be tagged with the homework name (“HW1”, for example).
- You must commit more than just the submission; it is good practice to commit whenever you add/change *anything*, whether that be adding one comment or a whole new class.
- Every assignment also has a written summary. You will submit those on Canvas to show your assignment is in GitHub and ready to be graded, but these written summaries must ALSO be in your folder in your repository.

When formatting and turning in your assignment be sure to follow: [Dr. Sprint's Coding Standards](#) which include the use of javadoc throughout your code. With each subsequent assignment, the documentation strictness and coding standards will increase.

Your code should be submitted via Canvas with your own private HW repo link and Dr. Wang, Abby, and Jon added as contributors. Your assignment repository should also contain a .pdf document describing the major design and development considerations encountered in implementing your assignment. The document should be named “Homework 1 Summary.pdf” and saved in the Summaries directory within the project repo. This document should be written in a narrative style and should include:

- A summary of the goal or purpose of the program in your own words.
- An overview of the general design you chose for your program.
- A UML Class Diagram that includes all classes that are part of your solution. Diagrams must be produced by a software tool and not hand drawn. I use MS Office Visio, dia, or app.diagrams.net for this.
- A description of any major design and/or programming issues, why these were the major issues, how you addressed them, and why you addressed them the way you did.
- A retrospective of what you would have done differently if you had more time.

IMPORTANT NOTE ON INTEGRITY

This is an individual assignment. Everything you turn in should be the result of keystrokes done by you. You should not consult nor use any existing classes related to Farkle or dice. You should not share your code nor look at the code of your classmates. It is ok to have generalized discussions about java and to have high-level design discussions about the classes, attributes, and methods necessary for the solution.

GRADING RUBRIC

Requirement	Points
Version checked in building and running	15
Summary document based on content and completeness	10
Following the Sprint coding style standards	5
Having a hand of 6 dice that tracks used and unused dice during the turn	10
Detecting a Farkle properly	15
Follows the crandallFarkle C++ User interface menu: Has the A through F Menu options Presents the unused dice to add to the meld and the dice in the meld Adds and removes dice to the meld Shows the current meld score	25
Allow player to end adding to the meld (Bank their points)	5
Showing the unused dice and the meld with its score after the user banks and the program quits	5
Whole GitHub Actions workflow running to completion	5
Homework #1 properly tagged as HW1 in the repository with README edited for name & HW1	5
Total Points	100

APPENDIX A: TERMINOLOGY AND CLARIFICATIONS

Note: These terminologies and clarifications are for the whole of Farkle, which is significantly more complex than what you're being asked to do in Homework #1. They are provided to help you ensure your classes, algorithms, and variable naming make sense in your program design.

Meld: is a collection of dice that a Zag Farkle player is putting together to score points after they rolled their dice. A Meld can be worth zero points if no dice Combos are found in it.

Combo: A set of dice that fit some set as described in the Scoring Combo Points Specification from Appendix B.

Banking: A player can only Bank if their current Meld is worth more than zero points. If Banking, then all Melds created during the player's turn have their points totaled up and the player's total score increases by this amount.

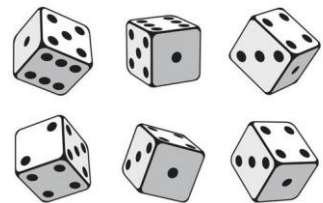
Rerolling: A player can only Reroll if their current Meld is worth more than zero points.

The quantity of dice rolled during a reroll is all unmelded dice (dice not added to the current meld) *and* any dice that did not get scored as part of a combo in the current Meld.

Quitting: If a player chooses the quit option, that means the entire game ends immediately, not just their turn

Hot Hand: If the player can reroll and they have zero dice left after combos are made in the current Meld, then that is a Hot Hand and they get 6 new dice to roll.

Farkle: When a player rolls or rerolls during their turn and the resulting hand of dice cannot make any Combos in the new Meld, which means the Meld's maximum score is zero, that is a Farkle and the total score for their whole turn is zero. This is true even if they did have prior success Melds during their current turn.



APPENDIX B: MELD AND COMBO SCORING SPECIFICATION FOR ZAG FARKLE

As you start building your project, the Meld class (if you have one) would be a phenomenal place for some good unit testing to show that it is working properly.

Within a Meld, one or more Combos might be found. Each Combo is worth points.

Note: No single die in a Meld can be used in two different combos. For example:

Meld: 1 1 1

Combos: Triple 1s - Total points 1000

The three dice with the value 1 are found in the Triple 1s combo, but do not *also* count for Single 1s.

Combo Name	Description	Example Dice	Points
Straight	One of each dice face value	1 2 3 4 5 6	1000
Three pairs	Three different pairs of dice face values	1 1 4 4 6 6	750
Triple 1s	Three dice showing 1s - special triple	1 1 1	1000
Triple #s	Basic triple: 100 * Dice face values	3 3 3	300
Triple #s +	Base triple + 100 * Extra dice face values	2 2 2 2	400
	Example with 5 x 6's - 600 + 600 + 600	6 6 6 6 6	1800
	Example with 4 x 5's - 500 + 500	5 5 5 5	1000
	Example with 6 x 1's - 1000 + 100 + 100 + 100	1 1 1 1 1 1	1300
Single 1s	Any single dice showing a 1	1	100
	Two 1s	1 1	200
Single 5s	Any single dice showing a 5	5	50
	Two 5s	5 5	100